

BINARY SEARCH

" Magic is believing in yourself.
If you can make that happen, you
can make anything happen. "

Johann Wolfgang Von Goethe

SATURDAY



Today's content

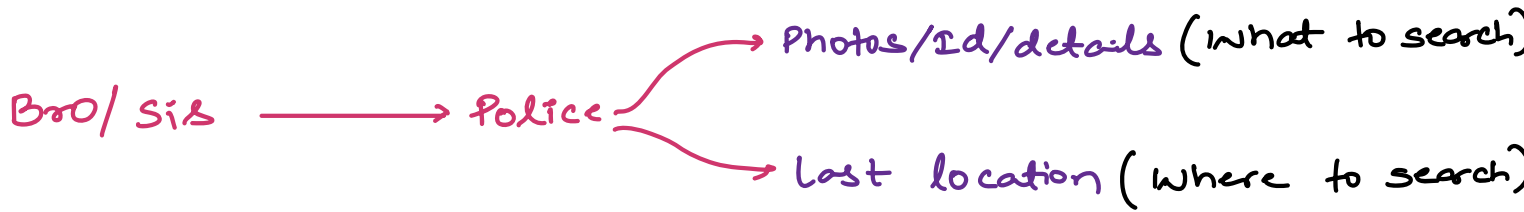
- Searching Basics
- Why mid at half?

Problems

- Search in sorted arr
- first occurrence of ele in sorted arr
- floor of a given value
- Finding local minima

Searching story

Target



Search space

Example

Target

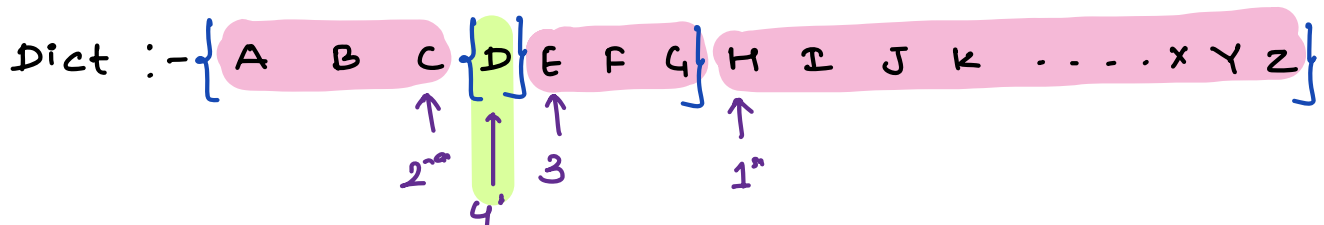
Search space

Word → { Dict / Newspaper / Books }

phone number → { Contacts list / Phone diary }

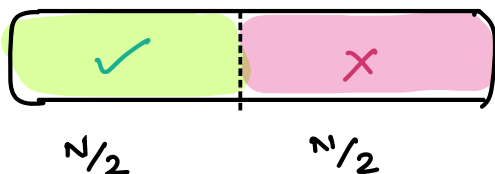
Obs': If search space is sorted, searching becomes easy

Search **dog** in dictionary

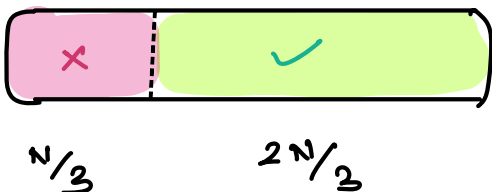


Why land at mid position?

Better



: Worst case : $\frac{N}{2}$ ele gets discarded



: Worst case : $\frac{N}{3}$ ele get discarded

When to apply Binary search?

→ After dividing the search space in two halves, if you can discard one half of the search space using **some condition** then you can apply BS

Note :- If you can't discard any search space, then you can't apply Binary search.

Q1. Given a **sorted arr[N]**, search if K is present or not?

arr[10] : { 3 6 9 12 14 19 20 23 25 27 } K=12
 0 1 2 3 4 5 6 7 8 9

Idea 1 :- Linear search

```
for (i=0; i<arr.length; i++) {  
    if (arr[i] == k) return true  
}
```

3

TC: O(N)
SC: O(1)

Idea 2: Binary search

Looking for k

Case - I



$arr[mid] = k$ return True

Case - II



$arr[mid] > k$: move to left
discard right

Case - III



$arr[mid] < k$: move to right
discard left

$arr[10] : \{ \underset{0}{3} \underset{1}{6} \underset{2}{9} \underset{3}{12} \underset{4}{14} \underset{5}{19} \underset{6}{20} \underset{7}{23} \underset{8}{25} \underset{9}{27} \}$ $k=12$

$\uparrow \uparrow$
lo hi

$\uparrow$
m

target = 12

$mid = \frac{(l+h)}{2}$

search space = [0 9]

lo hi mid

0 9 4

$arr[4] > 12$: move to left
 $h = m - 1$

0 3 1

$arr[1] < 12$: move to right
 $l = m + 1$

2 3 2

$arr[2] < 12$: move to right
 $l = m + 1$

3 3 3

$arr[3] = 12$: return true:

* Second ex

arr[10] : { 3 6 9 12 14 19 20 23 25 27 } k = 21
 0 1 2 3 4 5 6 7 8 9



$\frac{lo}{0}$	$\frac{hi}{9}$	$\frac{mid}{4}$
----------------	----------------	-----------------

arr[4] < 21 . move to right
 $l = m + 1$

5	9	7
---	---	---

arr[7] > 21 : move to left
 $h = m - 1$

5	6	5
---	---	---

arr[5] < 21 . move to right
 $l = m + 1$

6	6	6
---	---	---

arr[6] < 21 : move to right
 $l = m + 1$

7	6	→ Search space is exhausted
---	---	-----------------------------

return false

boolean search (int [] ar, int k)

l = 0 h = ar.length - 1

while (l ≤ h)

int m = $\frac{(l + h)}{2}$

if (ar[m] == k)

 | return true;
 3

else if (ar[m] < k) // discard left

 | // go to RHS
 l = m + 1
 3

else { // discard right

 | // move left
 h = m - 1;
 3

3

return false;

TC :

_____ : n ele

_____ $\frac{n}{2}$ ele

_____ $\frac{n}{4}$ ele

_____ $\frac{n}{8}$ ele

...

0

TC: $O(\log n)$

SC: $O(1)$

Q - Given a sorted array of elements, find first index of a given target.

arr[] =	-5	-5	-3	0	0	1	1	5	5	5	5	5	5	5	8	10	10	15	16
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

$\frac{k}{5}$

first index

7

-3

2

2

-1

Brute force: Linear search

Iterate & check if

arr[i] == k

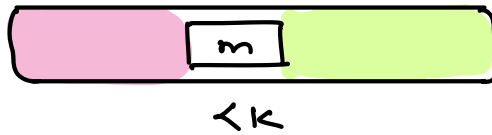
TC: $O(n)$

SC: $O(1)$

Idea 2 :- Binary search

K

Case - I



$ar[m] < K \rightarrow$ move to RHS
discard left

Case - II



$ar[m] > K \rightarrow$ move to LHS
discard right

Case - III



if ($ar[m] == K$)
|
| $ans = m;$
| $h = m - 1$ // move to left
|
3

arr[] =

-5	-5	-3	0	0	1	1	5	5	5	5	5	5	5	8	10	10	15	16
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

$\wedge$
 $\uparrow \quad \uparrow \quad \uparrow$
 $h \quad l \quad m$

l	h	mid
0	18	9

$ar[9] > 0$: move to left
 $h = m - 1$

0	8	4
---	---	---

$ar[4] = 0$: ans = 4
: $h = m - 1$

0	3	1
---	---	---

$ar[1] < 0$: move to right
 $l = m + 1$

2

3

2

$arr[2] < 0$: move to right
 $l = m + 1$

3

3

3

$arr[3] = 0$: $ans = 3$
 $h = m - 1$

3

2

→ search space exhausted

return $ans = 3$

```
int firstidx (int [] arr, int k)
```

```
    l = 0    h = arr.length - 1
```

```
    while (l ≤ h)
```

```
        int m =  $\frac{(l + h)}{2}$ 
```

TC: $O(\log n)$

SC: $O(1)$

```
        if (arr[m] == k)
```

```
            |   ans = m
```

```
            |   h = m - 1
```

```
            |   3
```

```
        else if (arr[m] < k) // discard left
```

```
            |   // go to rhs
```

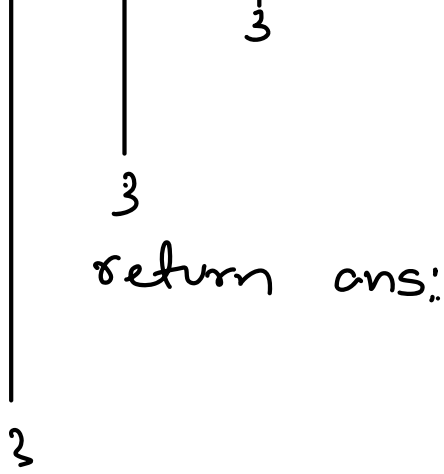
```
            |   l = m + 1
```

```
            |   3
```

```
        else { // discard right
```

```
            |   // move left
```

```
            |   h = m - 1;
```



TODO → { find the last occurrence of k }

* Unsorted array is given

01. HashMap = TC: $O(n)$
SC: $O(n)$

Usually we
give priority to
Time complexity

* 02. Sort the array - TC: $O(n \log n)$ + $O(\log n)$ = $O(n \log n)$
SC: $O(1)$
 ↳ Indexes will change
 ↳ can't do this

03. Linear search : TC: $O(n)$
SC: $O(1)$

10:14 pm → 10:24 pm

03. Given a sorted arr[], find floor of a given num K
greatest ele $\leq K$ in arr[]

arr[] = { -5 2 3 6 9 10 11 14 18 }
 0 1 2 3 4 5 6 7 8

K = 5 $\rightarrow$ ans = 3

K = 4 $\rightarrow$ ans = 3

K = 10 $\rightarrow$ ans = 10

K = -7 $\rightarrow$ ans $\rightarrow$ Nothing :

return Integer.MIN_VALUE

K : 1 $\rightarrow$ ans = -5

K = 24 $\rightarrow$ ans = 18

arr[] = { -5 2 3 6 9 10 11 14 18 } K = 12
 0 1 2 3 4 5 6 7 8

* Ideal : Linear search , ans = $-\infty$

i	arr[i]	ans
0	-5	ans = -5
1	2	ans = 2
2	3	ans = 3
3	6	ans = 6
4	9	ans = 9

5 10 ans = 10

6 11 ans = 11

7 14 ele > k
return ans

Idea 2 : Binary search

Case - I 
 = k

arr[mid] == k return
arr[mid]

Case - II 
 < k

arr[mid] < k : move to right
ans = mid
l = mid + 1

Case - III 
 > k

arr[m] > k : go to left
h = mid - 1

arr[9] = { -5 2 3 6 9 10 11 14 18 } K = 12
 0 1 2 3 4 5 6 7 8

↑ ↑ ↑
h l m

l h mid
— — —

0	8	4	$arr[4] < 12$	$ans = 9$	$l = m + 1$
5	8	6	$arr[6] < 12$	$ans = 11$	$l = m + 1$
7	8	7	$arr[7] > 12$	$h = m - 1$	
7	6	 Exhausted search space			

Ans = 11

```
int floor (int [] arr , int k)
```

$l = 0$, $h = arr.length - 1$, $ans = \text{Integer.MIN_VALUE}$

```
while (l ≤ h)
```

```
    int m =  $\frac{(l + h)}{2}$ 
```

TC: $O(\log n)$

SC: $O(1)$

```
    if (arr[m] == k)
```

```
        return ans[m];
```

```
    else if (arr[m] < k) // discard left
```

```
        // go to RHS
```

```
        ans = arr[m]
```

```
        l = m + 1
```

```
    else { // discard right
```

```
        // move left
```

```
        h = m - 1;
```

3
return ans;

3

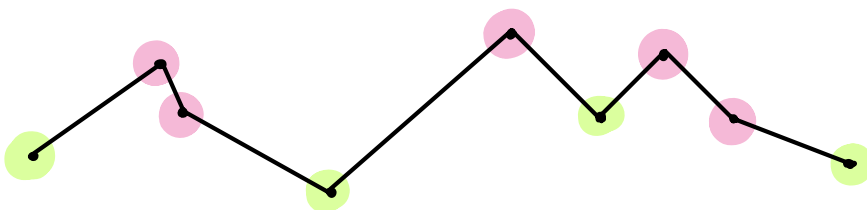
04. Given unsorted arr[] distinct elements, return any local minima. An element is said to be local minima, if it is less than its adjacent elements.

Eg:- arr[] = { 9 8 7 3 6 4 1 5 2 }

0 1 2 3 4 5 6 7 8

Local minima $arr[i-1] > arr[i] < arr[i+1]$

Edge cases { $arr[0] < arr[1]$
 $arr[n-1] < arr[n-2]$



Eg:- $ar[8] = \{ \begin{array}{c} 9 \ 8 \ 7 \ 3 \ 6 \ 4 \ 1 \ 5 \ 2 \\ 0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \end{array} \}$ Ans = 3 or 1 or 2

Idea 1:- Linear search

Iterate on $ar[]$, compare $ar[i]$ with

$ar[i-1]$ & $ar[i+1]$ TC: $O(n)$
SC: $O(1)$

Idea 2: Binary Search

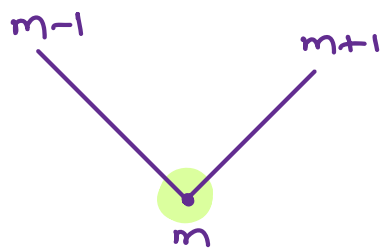
Target: Find local minima

Search space: 0 to $n-1$

How to reduce search space?

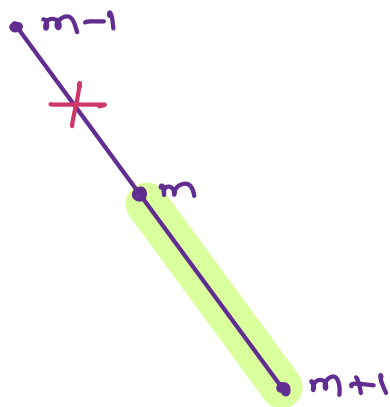


Case - I



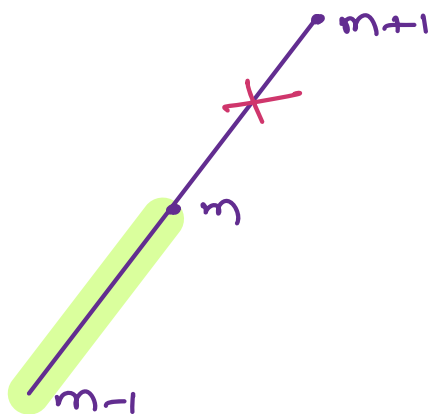
return $ar[m]$

Case II



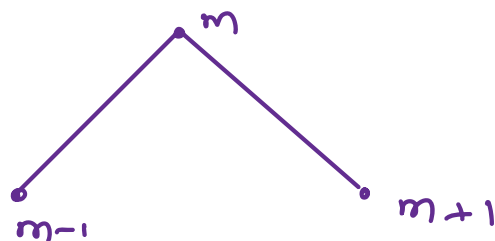
move to RHS

Case III



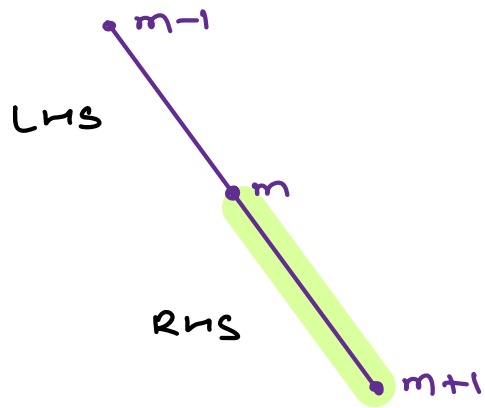
Move to LHS

Case IV

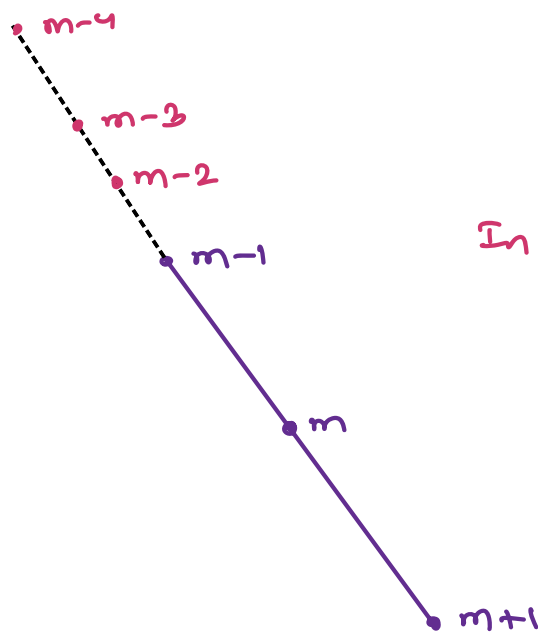


Move on to LHS

* Analysing case-II

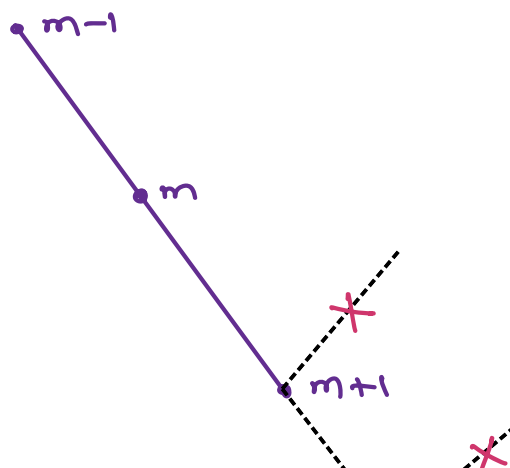


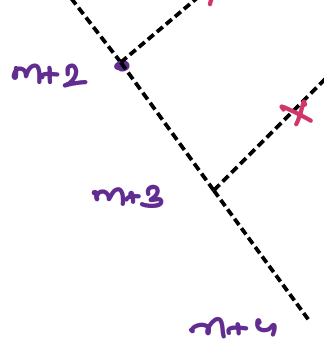
* Exploring LHS



In worst case scenario:
no answer on LHS

* Exploring RHS





Pseudocode

if ($ar.length == 1$) return $ar[0]$

if ($ar[0] < ar[1]$) return $ar[0]$

if ($ar[n-1] < ar[n-2]$) return $ar[n-1]$;

$l = 1$ $h = n - 2$

while ($l \leq h$)

$mid = (l + h) / 2$;

 if ($ar[mid] < ar[mid-1]$ && $ar[mid] < ar[mid+1]$)

 |
 return $ar[mid]$
 }

 else if ($ar[mid] < ar[mid-1]$ && $ar[mid] > ar[mid+1]$)

 |
 $l = mid + 1$
 }

else {

h = mid - 1

3

3

3

eg:- ar[8] = { 9 8 7 3 6 4 1 5 2 }

0 1 2 3 4 5 6 7 8

↑
rh
↑
m

ar[4]

3 < 6 > 4

9 > 8 > 7

$$\frac{0+3}{2} = 1$$

$$\frac{2+3}{2} = 2$$

$l = m + 1$ { $\frac{ar[m] < ar[m-1]}{ar[m] > ar[m+1]}$

$\boxed{7} < 8$

$\boxed{7} > 3$

Doubts

arr = { 3, 0, 6, 1, 5 }

h-index = 3

Sort = { 0, 1, 3, [✓]5, [✗]6 }

Max = 6 > arr.length ✗

h-index = 6

Max = 5 = don't have 5 paper citations > 5

$$\text{max} = 3 = n - 1 - i + 1 \Rightarrow \boxed{n - i}$$

$$\underline{2 \rightarrow 4} \rightarrow \textcircled{3}$$

Idea = Arrays Sort